

Benchmarking and Performance Analysis of LLM Inferencing Systems

Saksham Rathi

Systems for LLMs
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

Contents

1. The Core Trade-off
2. Chunked Hash Tree
3. RL Problem Formulation
4. Policies

The Core Trade-off



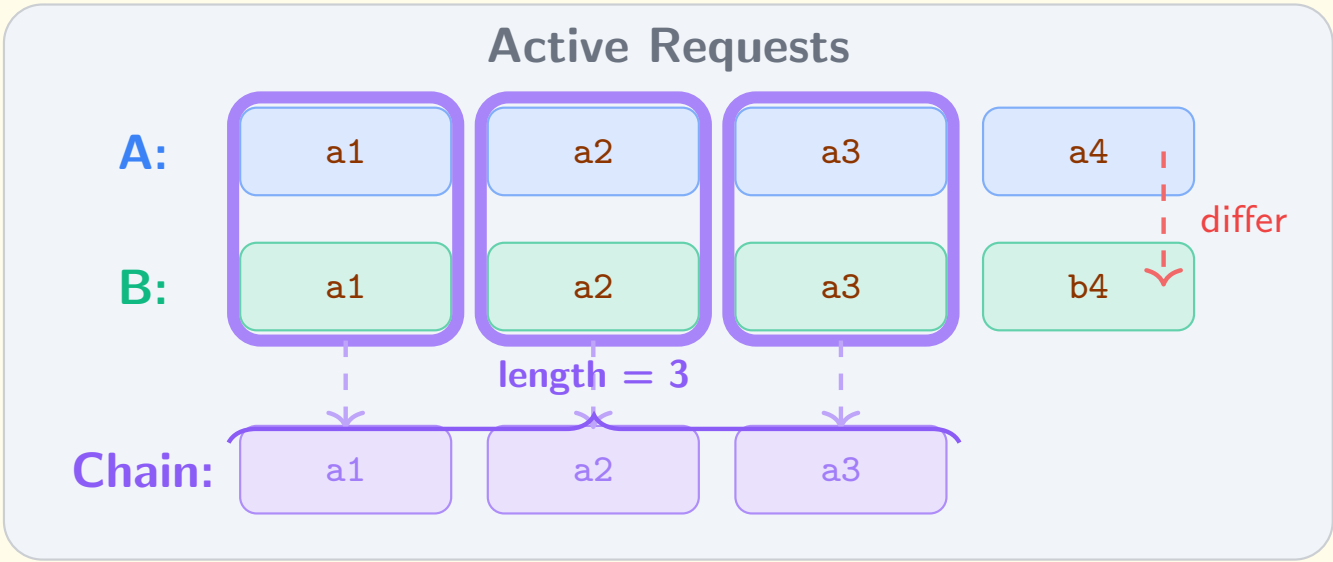
- Larger Batches $\rightarrow$ Higher GPU Utilization
- Larger Shared Prefix $\rightarrow$ Better Temporal and Spatial Locality
- Need to ensure the prefix is the same across **all** the requests

Changes to Chunked Hash Tree



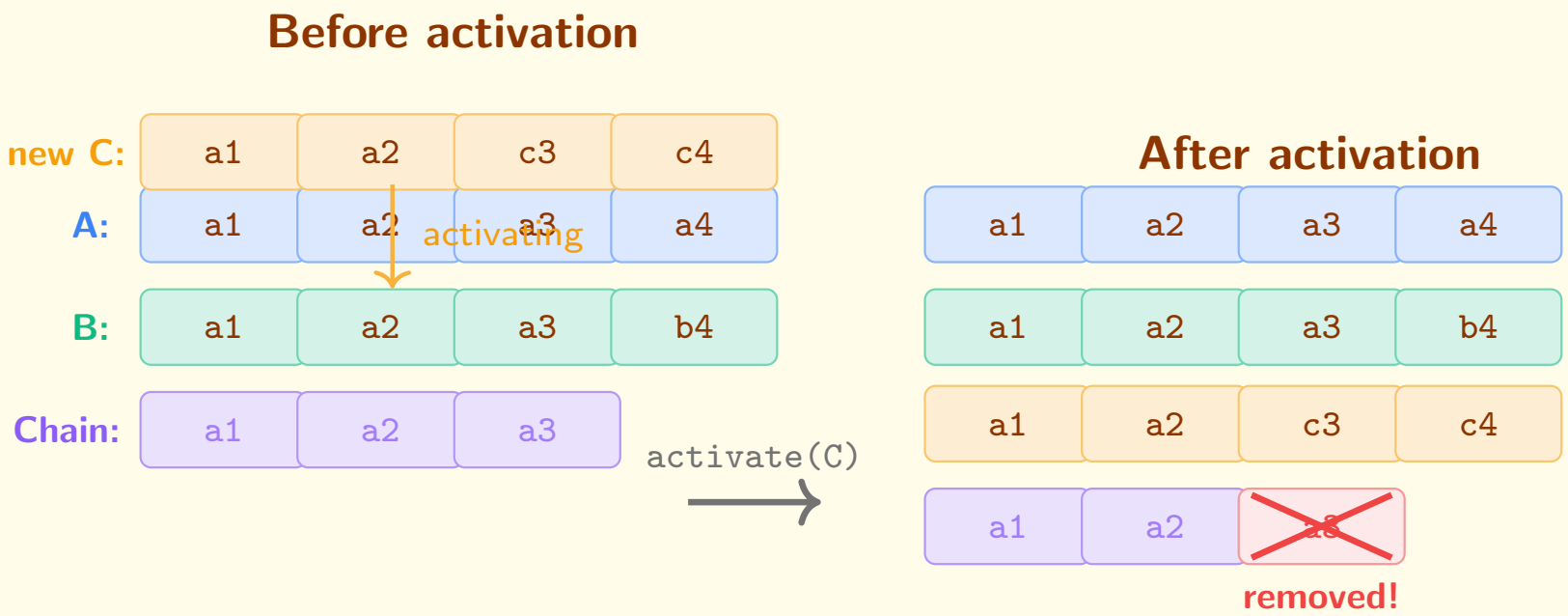
- Previously, we used to find the cost of adding a request to the working set (in terms of the newer chunks)
- Now we need information regarding the shared prefix chunks of all the active requests, along with changes to this upon adding a new request

Shared Prefix Chain



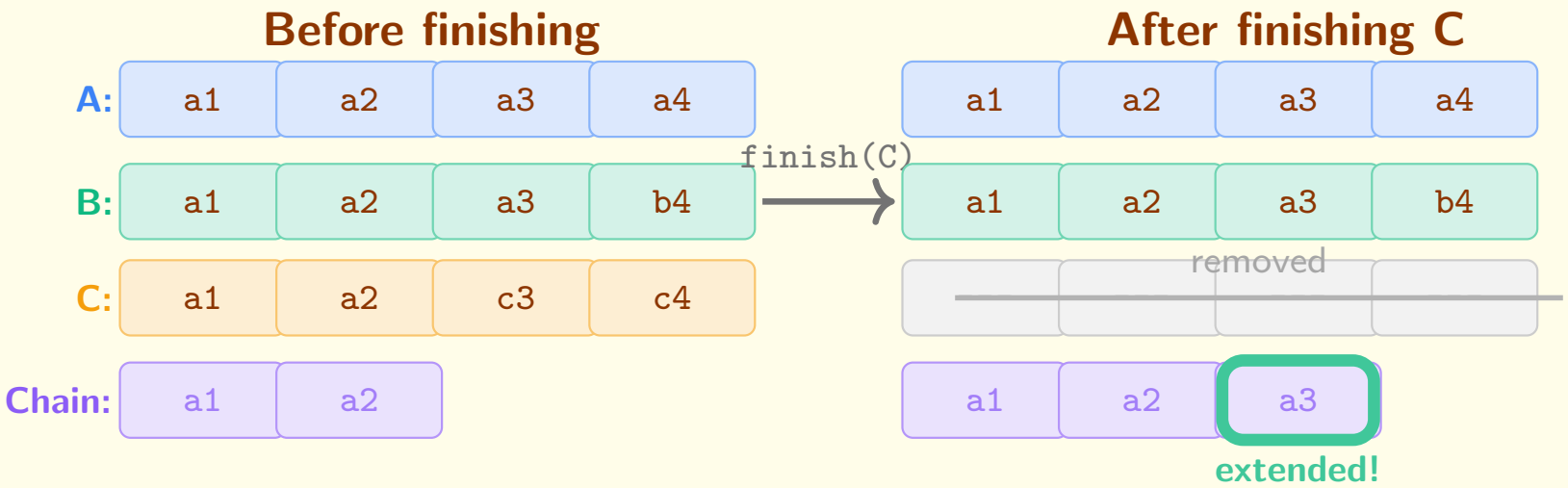
Shared Prefix Chain: A vector of hash blocks identical across all active requests.

Effect of `activate_request()` – Chain Shrinks



Why shrinks: C has c3 at level 2, but chain holds a3. Scan forward; stop at first mismatch.

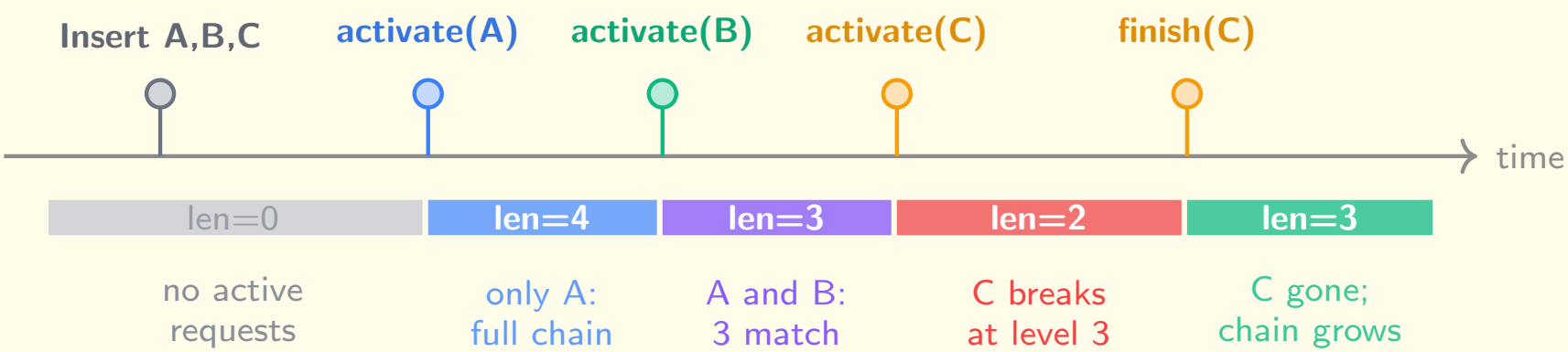
Effect of `finish_request()` – Chain Grows



Why grows: C was blocking a3: it had c3 at level 2 instead. Now only A and B remain; both share a3 at level 2 $\Rightarrow$ scan forward and extend.

Systems for LLMs
Saksham
The Core Trade-off
Chunked Hash Tree
RL Problem Formulation
Policies

Shared Prefix Chain – Full Lifecycle



Systems for LLMs
Saksham
The Core Trade-off
Chunked Hash Tree
RL Problem Formulation
Policies

State

- **B:** current batch size
- $\mathbf{C_B}$: number of chunks shared before adding a new request
- $\mathbf{C_A}$: number of chunks which will remain shared after adding the new (best) request
- $\mathbf{R_W}$: number of requests waiting at the end of C_A

State

$s_t \in (B, C_B, C_A, R_W)$

Action

- **ADD:** pop and activate the best request from the chunked hash tree
- **SKIP:** break the batch construction loop, dispatch the batch to GPU

Action

$a_t \in (\texttt{ADD}, \texttt{SKIP})$

Reward

- Negative decode wall step time of the previous batch
- Delivered once per batch

Reward

$$r_t = -t_{\texttt{exec}}^{(t-1)}$$

Policy 1: Q-Learning Scheduler



- Trial-and-error learning method
- The system tries actions (ADD or SKIP), observes rewards and gradually learns which action works best in each situation.
- **Goal:** "In this situation, which action gives the best long-term outcome?"

Policy 1: Q-Learning Scheduler



State Discretization (Making the problem manageable)
The real system has continuous values. We convert them into buckets (like "low", "medium", "high").

$$\tilde{s}_t = (\tilde{B}, \widetilde{C_B - C_A}, \tilde{R}_W)$$

Where:

- B = batch size
- $C_B - C_A$ = chunk difference (loss signal)
- R_W = number of waiting requests

Intuition: Instead of remembering every exact number, we group similar situations together.

Policy 1: Q-Learning Scheduler



Q-Function (What is being learned?)

$$Q(s_t, a_t)$$

This is a table storing:

$$Q : (\tilde{s}, a) \to \mathbb{R}$$

For each state and action, store a number that estimates "how good this action is in this situation".

Policy 1: Q-Learning Scheduler



The system must balance:

- Try new actions (exploration)
- Use what it already learned (exploitation)

Epsilon-Greedy Policy

$$a_t = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg\max_a Q(s_t, a) & \text{otherwise} \end{cases}$$

- With probability ϵ, try something random
- Otherwise, choose the action with highest learned value

Policy 1: Q-Learning Scheduler



Intuition: Early on, explore a lot. Later, trust what you learned.

Exploration Decay

$$\epsilon_t = \begin{cases} \epsilon_{\text{start}} - (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \frac{t}{T_{\text{decay}}} & t < T_{\text{decay}} \\ \epsilon_{\text{end}} & \text{otherwise} \end{cases}$$

Policy 1: Q-Learning Scheduler



How Learning Happens

After taking action a_t:

- Observe reward r_t
- Update the table

Update Rule

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_t - Q(s_t, a_t)\right)$$

- If reward is better than expected $\rightarrow$ increase value
- If worse $\rightarrow$ decrease value

α = learning rate (how fast we adjust)

Policy 1: Q-Learning Scheduler



Overall Objective

$$\max_{\pi} \ \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

Find a policy π that maximizes long-term rewards.
Not just immediate reward, we care about the future too.

Policy 2: Contextual Bandit (UCB)



- This is a simpler learning strategy.
- It does NOT consider long-term future.
- **Goal:** "Which action works best right now in this situation?"

$$\hat{s}_t = \text{bin}(B, C_B - C_A, R_W)$$

We still bucket the state.

Policy 2: Contextual Bandit (UCB)



Statistics Stored

$$N(\hat{s}, a) \quad \text{(how many times tried)}$$

$$\bar{r}(\hat{s}, a) = \frac{\sum r}{N(\hat{s}, a)}$$

Track average reward of each action.

Policy 2: Contextual Bandit (UCB)



Upper Confidence Bound (UCB)

$$\text{UCB}(\hat{s}, a) = \bar{r}(\hat{s}, a) + c\sqrt{\frac{\log T}{N(\hat{s}, a)}}$$

Intuition:

Two parts:

- Exploit: choose high average reward
- Explore: boost rarely tried actions

If an action was rarely tried, its uncertainty is large $\rightarrow$ encourage exploration.

Policy 2: Contextual Bandit (UCB)



Policy

$$a_t = \arg\max_a \mathrm{UCB}(\hat{s}_t, a)$$

Meaning: Choose the action with highest optimistic estimate.

Policy 3: Heuristic Scheduler



- This policy does NOT learn.
- It uses manually designed rules.

Chunk Loss $\Delta C = C_B - C_A$

Interpretation:

- If ΔC small $\rightarrow$ memory accesses remain similar
- If large $\rightarrow$ too much loss in temporal and spatial locality

Policy 3: Heuristic Scheduler



Decision Logic

1. If no batch $\rightarrow$ ADD
2. If no chunk loss $\rightarrow$ ADD
3. If batch small $\rightarrow$ tolerate slightly more loss
4. If batch large $\rightarrow$ be stricter
5. If many waiting requests $\rightarrow$ prioritize ADD
6. Otherwise $\rightarrow$ SKIP



Thank You